

PLANT DISEASE DETECTION USING DEEP LEARNING

A Project Report Submitted in Partial Fulfillment of the Requirements for the Award of the
Degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by:

CHITTELA VENKATA SIVA PRASAD

(Reg.No: 23AR1F0005)

Under the esteemed guidance of

Mrs.M.APARNA, M. Tech

Assistant Professor



DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

TIRUMALA INSTITUTE OF TECHNOLOGY AND SCIENCE

COLLEGE

(Affiliated to J.N.T. University, KAKINADA, Approved by AICTE, Accredited by NAAC)

Satuluru, Guntur Road, Narasaraopet (MD), Guntur (DT),

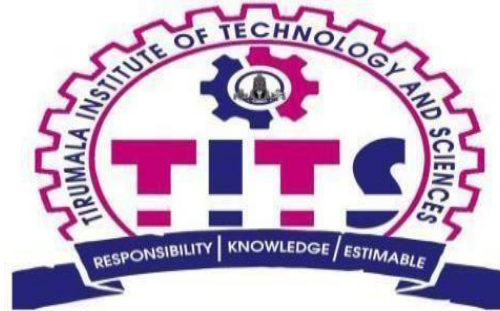
Andhra Pradesh-522601

2023-2025

TIRUMALA INSTITUTE OF TECHNOLOGY AND SCIENCE COLLEGE

(Affiliated to J.N.T. University, KAKINADA, Approved by AICTE, Accredited by NAAC)

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS



CERTIFICATE

This is to certify that the project report entitled **“PLANT DISEASE DETECTION USING DEEP LEARNING”** submitted By CHITTELA VENKATA SIVA PRASAD(23AR1F0005) Partial Fulfillment for the Award of the Degree of **“MASTER OF COMPUTER APPLICATIONS”** is a record bonafide work carried out by us under the guidance and supervision during the academic year 2023-2025 and found worthy of acceptance to requirements of the university.

Internal Guide

Mrs.M.APARNA,M.Tech,Ph.D
Assistant Professor

Head of the Department

Dr. K. PRASADARAO,M.Tech,Ph.D
Professor

External Examiner

TIRUMALA INSTITUTE OF TECHNOLOGY AND SCIENCE COLLEGE

(Affiliated to J.N.T. University, KAKINADA, Approved by AICTE, Accredited by NAAC)

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS



DECLARATION

I here by declare that this dissertation entitled **“PLANT DISEASE DETECTION USING DEEP LEARNING”** being submitted to the Department Of **MASTER OF COMPUTER APPLICATIONS** Accredited by AICTE, affiliated to JNTU, KAKINADA for the award of Mater of Computer Applications is a record of bonafide work done by me and it has not been submitted to anyother Institute or University of the award of any degree or Prize.

Place:

Date:

SUBMITTED BY:

CHITTELA VENKATA SIVA PRASAD

23AR1F0005

ACKNOWLEDGEMENT

I express our deep sense of gratitude to **Dr. NALABOTHU VENKATA RAO, B.V.Sc.**, Chairman of **TIRUMALA INSTITUTE OF TECHNOLOGY AND SCIENCE COLLEGE** Narasaraopet, for creating excellent academic atmosphere and providing good infrastructural facilities to us.

I express our deep felt gratitude to Our Director **Dr. Y. V. Narayana, B. Tech, M. Tech, Ph.D** of **TIRUMALA INSTITUTE OF TECHNOLOGY AND SCIENCE COLLEGE** for helping us providing all necessary facilities, support and encouragement in successful completion of the project.

I am graceful to our principal Prof. **Dr. K. PRASAD RAO, B. Tech, M. Tech, Ph.D** for providing us all the necessary facilities for the completion of this project in specified time.

I express our sincere gratitude to **Dr. K. PRASADA RAO, B.Tech. M. Tech, Ph.D.** Professor and Head of the Department of Computer Science & Engineering, for his support and encouragement during the period of project work.

I express our sincere thanks to my guide **Mrs.M.APARNA, M.Tech, Ph.D** Assistance Professor for his valuable guidance and constant encouragement which enabled us to accomplish our project successfully in time. His vast experience, profound knowledge and willingness have been a constant source of inspiration for us throughout this project work.

I express our special thanks to all the library staff of **TIRUMALA INSTITUTE OF TECHNOLOGY AND SCIENCE COLLEGE**, for providing the necessary library facilities. Last but not the least, we express our heartfelt thanks to all our friends in our endeavour to complete the project successfully.

CHITTELA VENKATA SIVA PRASAD
23AR1F0005

ABSTRACT

The global rise in population has led to a shortage of raw materials and food supplies. The agricultural sector has become the primary and most vital source to overcome this particular constraint. However, the industry itself is facing the challenge of pests and various crop diseases. Battling this has been the significant focus of the sector for decades. Still, due to the technology gap that existed earlier, there existed a constraint on identifying the diseased crop on a massive scale. Nevertheless, today, with the improvement of technologies such as drones, IoT devices, and higher processing speeds combined with data analysis and machine learning, the problem of identification can be resolved quickly. The proposed model using CNN was trained using images from plant village dataset and attained an accuracy of 94.87% in identifying the diseased plants with the help of image processing by Tensorflow.

Plant pests and diseases are a significant threat to almost all major types of plants and global food security. Traditional inspection across different plant fields is time-consuming and impractical for a wider plantation size, thus reducing crop production. Therefore, many smart agricultural practices are deployed to control plant diseases and pests. Most of these approaches, for example, use vision-based artificial intelligence (AI), machine learning (ML), or deep learning (DL) methods and models to provide disease detection solutions. However, existing open issues must be considered and addressed before AI methods can be used. In this study, we conduct a systematic literature review (SLR) and present a detailed survey of the studies employing data collection techniques and publicly available datasets. To begin the review, 1349 papers were chosen from five major academic databases, namely Springer, IEEE Xplore, Scopus, Google Scholar, and ACM library. After deploying a comprehensive screening process, the review considered 176 final studies based on the importance of the method. Several crops, including grapes, rice, apples, cucumbers, maize, tomatoes, wheat, and potatoes, have tested mainly on the hyperspectral imagery and vision-centered approaches.

INDEX

S.NO	CONTENT	PAGE NO
1	INTRODUCTION	1-12
	Introduction	8-10
	Objective of the project	11
	Literature Survey	12
2	SYSTEM ANALYSIS AND DESIGN	13-
	Existing System	13
	Drawbacks	13
	Proposed System	14
	Advantages	14
	System Architecture	15
	Data Flow Diagram	16
	System Modules	17-18
	Feasibility Study	19
	Platform	20
	Software Requirements	20
	Hardware Requirements	20
3	CONCEPTS AND METHODS	21-24
	Concepts	21-22
	Algorithms	23
	Methodology	24
4	UML DIAGRAMS	25-29

5	SOFTWARE ENVIRONMENT	30-36
6	SYSTEM TESTING	37-40
	Test Strategy	38
	Test Cases	39-40
7	IMPLEMENTATION	41-55
	Sample Code	41-49
	Screenshots	50-54
	Results and comparison	54-55
8	FUTURE ENHANCEMENTS	56
9	CONCLUSION	57
10	BIBLIOGRAPHY	58
	PLAGARISM REPORT	59-61

INTRODUCTION

INTRODUCTION

Identification of the plant diseases is the key to preventing the losses in the yield and quantity of the agricultural product. The studies of the plant diseases mean the studies of visually observable patterns seen on the plant. Health monitoring and disease detection on plant is very critical for sustainable agriculture. It is very difficult to monitor the plant diseases manually. It requires tremendous amount of work, expertise in the plant diseases, and require the excessive processing time. Hence, image processing and Machine learning techniques are used for the detection of plant diseases. Disease detection involves the steps like image acquisition, image pre-processing, image segmentation, feature extraction and classification. Crop growth is basically determined by the climate variables in the environment and by the amounts of water and fertilizers that can be supplied by irrigation. Therefore, crop growth can be managed by controlling these variables. This makes a greenhouse ideal for growing crops because it is enclosed; and these variables can be manipulated for optimal plant growth and development. Having ideal climatic conditions present in the greenhouse might be optimal for plant growth but they also favor the proliferation of pests and diseases. This is the case with the disease caused by the fungus *Botrytis cinerea* (i.e. [Botrytis](#)), which thrives on humidity in the air. The lack of ventilation in Mediterranean greenhouses means that condensation in production processes under plastic is ever-present and, consequently, can cause severe damage.

Plant pathogens and pests cause substantial reduction in plant production depending on adverse seasonal and environmental conditions leading to economic and social losses. Contemporary pests and pathogen management depend profoundly on pesticide application, for example, herbicides, fungicides, and insecticide.

According to the United Nations' global goals, zero hunger is approximated to be accomplished by 2030. Early plant disease detection and classification significantly increase food security to attain the target. According to the World Health Organization (WHO), as of 2021, the total global population is approximately 7.837 billion, indicating that the demand for food increases along with the global population. Prioritizing food production is vital to addressing the issue of global hunger since the UN reported that global hunger numbers rose to as much as 828 million in 2021.

Farmers have widely used chemicals, for instance, insecticides and herbicides, to manage plant diseases and pests and increase crop yields. In the short run, these chemicals increase plant yields without decreasing crop quality. However, in the long run, for instance, in terms of health, chemicals such as herbicides pollute the ground where they are used and the environment in general. This is because of their chemical toxicity, which poses severe health risks and causes nearly three hundred thousand deaths annually, as demonstrated.

Detecting plant diseases early can minimize the use of hazardous chemicals in plant growth and protection. For example, IoT methods for detecting and diagnosing plant health status have been proposed, as have recent artificial intelligence (AI) methods such as deep and machine learning and image processing (IP) centered disease detection. Various methods have been presented in recent years to appreciate AI's increased agricultural application, for example, deep learning (DL) approaches in building advanced AI models for plant disease detection.

Despite advances in AI-based methods, plant disease detection in natural environments remains an issue. Drones are being used to monitor plant health to track diseases. As a result, compared to digital cameras, these semi- or autonomous aircraft provide robust and dependable vision methods that can be used on various crops.

This study provides a detailed overview of the most recent disease identification and PDD approaches, focusing on the most used AI (ML and DL) and IP algorithms for disease identification. Moreover, it systematically evaluates the limitations, strengths, and significant traits of these methods in real-world applications. A search of five major academic research databases, namely Springer, IEEE Xplore, Scopus, Google Scholar, and ACM, yielded 1349 papers for this review. Some keywords that were used during the search include “disease classification,” “disease identification,” “crop disease detection,” “PDD,” “ML,” “DL,” and “IP.” According to the significance of the recent surveys, the prominence of the approach, performance in PDD, and the used datasets, 176 studies were selected for the study.

OBJECTIVES OF THE PROJECT

The objectives of the project "Plant Disease Detection using Deep Learning" are as follows:

Primary Objectives

1. **Develop a Deep Learning Model:** Design and train a deep learning model that can accurately detect plant diseases from images of plant leaves or other parts.
2. **Improve Disease Detection Accuracy:** Achieve high accuracy in detecting plant diseases, reducing false positives and false negatives.
3. **Identify Disease Types:** Develop a model that can identify specific types of plant diseases, such as fungal, bacterial, or viral infections.

Secondary Objectives

1. **Image Preprocessing:** Develop techniques for preprocessing images of plant leaves or other parts to enhance model performance.
2. **Dataset Creation:** Create a large dataset of images of plant diseases to train and test the deep learning model.
3. **Model Optimization:** Optimize the deep learning model for deployment on various platforms, such as desktop computers, mobile devices, or embedded systems.
4. **User Interface Development:** Develop a user-friendly interface for farmers, researchers, or other stakeholders to interact with the plant disease detection system.

Tertiary Objectives

1. **Improve Crop Yields:** By detecting plant diseases early, reduce crop losses and improve yields.
2. **Reduce Pesticide Use:** By identifying specific diseases, reduce the use of pesticides and other chemicals.
3. **Enhance Food Security:** Contribute to global food security by improving crop yields and reducing losses due to plant diseases.
4. **Support Sustainable Agriculture:** Promote sustainable agriculture practices by providing a tool for farmers to detect and manage plant diseases in an environmentally friendly way.

LITERATURE SURVEY

The general process of using traditional image recognition processing technology to identify plant diseases is shown in 1. Dubey and Jalal [3] used the K-means clustering method to segment the lesions regions, and combined the global color histogram (GCH) color coherence vector (CCV) local binary pattern (LBP), and completed local binary pattern (CLBP) was used to extract the color and texture features of apple spots, and three kinds of apple diseases were detected and identified based on improved support vector machine (SVM), and the classification accuracy reached 93%.

THE HISTORY OF DEEP LEARNING

Deep Learning (DL) is a subclass of Machine Learning (ML), It was introduced in 1943 [18] and then went into three stages of development. The first generation of neural network-MCP (1943~1969): originated in 1943, is a linear model, can only deal with linear classification problems.

The second generation of neural network-back propagation (BP) (1986~1998): Hinton invented the BP algorithm suitable for multi-layer perceptron (MLP) in 1986 and adopted sigmoid function for nonlinear mapping, which effectively solved the problem of nonlinear classification and learning. This method caused the second upsurge of neural networks. However, in 1991, the BP algorithm was pointed out that there was a gradient vanishing problem. The third generation neural network-DL(2006-present): In 2006, Hinton gradient disappeared in the deep web training are put forward in this problem solution, but because there is no special effective experimental verification and no attention. It was not until 2011 that the ReLU activation function (the activation function that can effectively restrain the gradient disappeared problem) was put forward, then enter the outbreak period in 2012, in the famous Image Net image recognition contest, the Hinton team used a deep learning model-AlexNet to win, and far more than the second method (SVM). Since then CNN has attracted the attention of many researchers.

After the introduction of AlexNet [19], the DL architecture began to evolve over time as shown in VI. Many advanced DL models/architectures were used for image detection, segmentation, and classification, and these architectures were successively applied to plant disease detection.

SYSTEM ANALYSIS AND DESIGN

SYSTEM ANALYSIS AND DESIGN

EXISTING SYSTEM

The Leaf shape description is that the key downside in leaf identification. Up to now, several form options are extracted to explain the leaf form. However, there's no correct application to classify the leaf once capturing its image and identifying its attributes, however. In plant leaf classification leaf is classed supported its completely different morphological options.

A number of the classification techniques used are:

- ☐ Fuzzy logic
- ☐ Principal component Analysis
- ☐ k-Nearest Neighbors Classifier

DRAWBACKS

Low accuracy

Accuracy refers to the proportion of correctly classified instances among the total instances. In the context of deep learning models for plant disease detection, low accuracy means that the model is making a significant number of incorrect predictions.

High complexity

Complexity refers to the intricacy or sophistication of a system, in this case, the deep learning model used for plant disease detection. High complexity means that the model architecture, training process, or inference procedure is intricate and difficult to understand or manage.

Detection of images is classified with any noise

Noise refers to irrelevant or erroneous information present in the input data. In the context of image classification for plant disease detection, noise can manifest as unwanted artifacts, distortions, or variations in the images that interfere with accurate classify.

PROPOSED SYSTEM

The main purpose of proposed system is to detect the diseases of plant leaves by using feature extraction methods where features such as shape, color, and texture are taken into consideration. Convolutional neural network (CNN), a machine learning technique is used in classifying the plant leaves into healthy or diseased and if it is a diseased plant leaf, CNN will give the name of that disease. Suggesting remedies for disease is made which will help in growing healthy plants and improve the productivity. First the images of various leaves are acquired using high resolution camera to get the better results & efficiency. Then image processing techniques are applied to these images to extract useful features which will be required for further analysis. The basic steps of the system are summarized as:

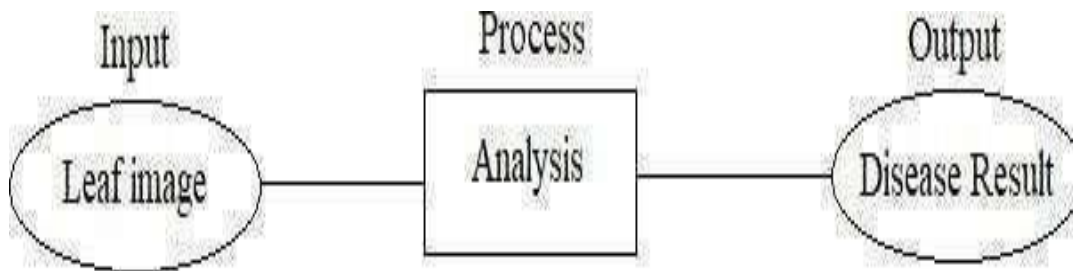


Fig 1.1 Proposed System.

The advantages of proposed algorithm are as follows:

- Use of estimators for automatic Initialization of cluster centers so there is no need of user input at the time of segmentation.
- The detection accuracy is enhanced with proposed algorithm.
- Proposed method is fully automatic while existing methods require user input to select the best segmentation of input image.
- It also provides environment friendly recovery measures of the identified disease

SYSTEM ANALYSIS

SYSTEM ARCHITECTURE

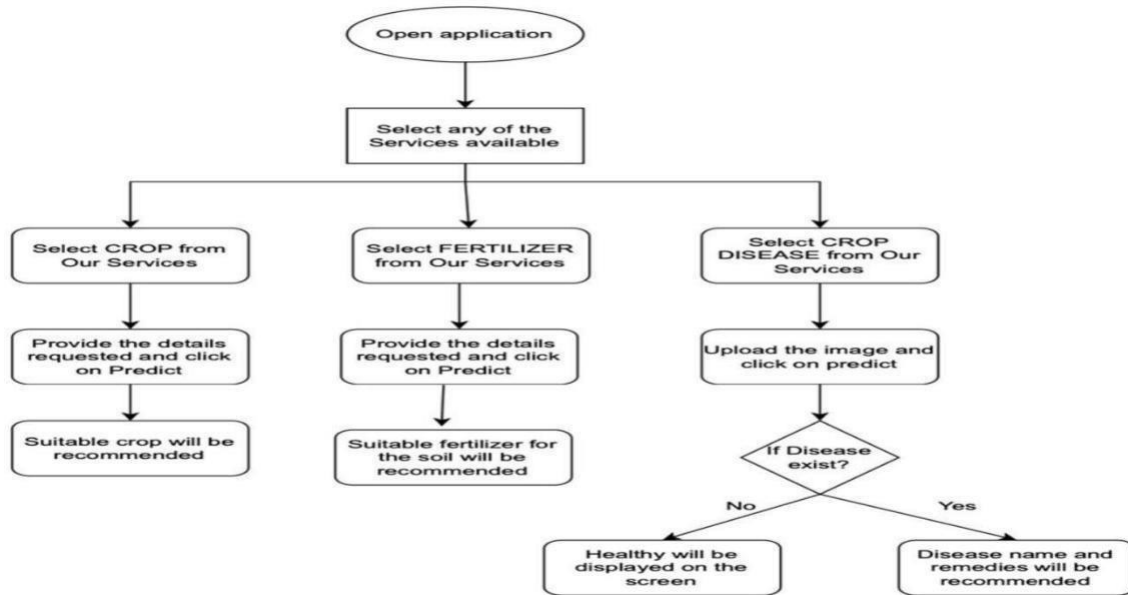


Fig : System Architecture

Fig. shows the system architecture of our proposed model. It is a mobile app that consists of three modules- crop recommendation, fertilizer recommendation, and plant disease classification. The crop recommendation module recommends the crop based on the values of the different parameters given by the user. The fertilizer recommendation module suggests how to improve the fertility of the soil. The plant disease classification module helps the farmer to identify the disease.

DATA FLOW DIAGRAM

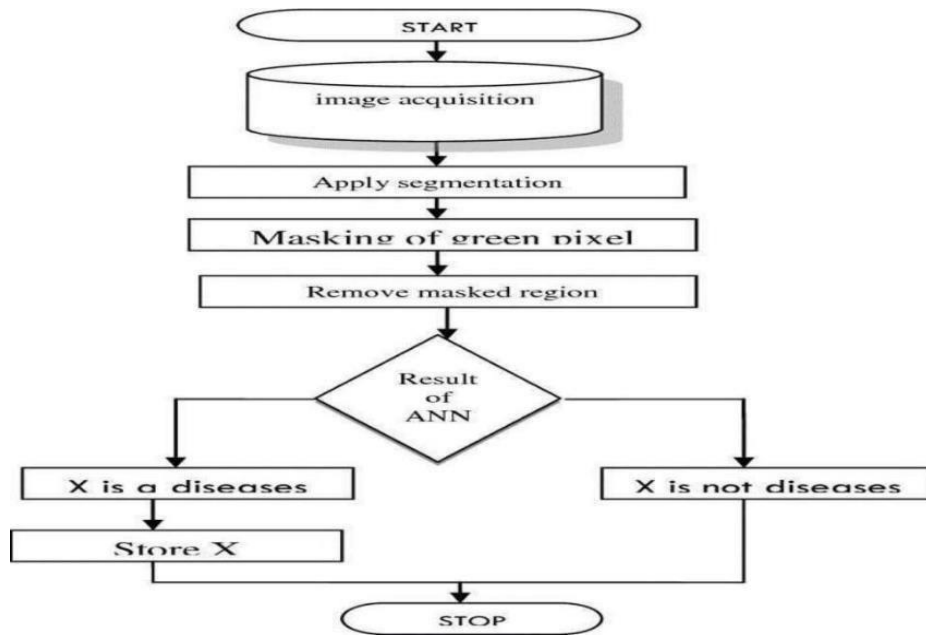


Fig : Flowchart for crop recommendation

A data flow Diagram is a graphical representation of the “flow” of data through an information system, modelling its process aspects. A DFD is often used as a preliminary step to create an overview of the system without going into detail, which can later be.

SYSTEM MODULES

The "Plant Disease Detection Using Deep Learning" involves several modules to accomplish the task effectively. Here's how each module contributes to the overall process:

Image Acquisition

1. Image Capture: Capture images of plant leaves or other parts using cameras, smartphones, or other devices.
2. Image Storage: Store images in a database or file system for further processing.

Image Preprocessing

1. Image Resizing: Resize images to a uniform size for processing.
2. Image Normalization: Normalize image pixel values to improve model performance.
3. Noise Reduction: Apply filters to reduce noise in images.
4. Data Augmentation: Apply techniques such as rotation, flipping, and cropping to increase dataset size.

Deep Learning Model

1. Convolutional Neural Network (CNN): Design and train a CNN to detect plant diseases from images.
2. Transfer Learning: Use pre-trained models such as VGG16, ResNet50, or InceptionV3 as a starting point for the CNN.
3. Model Training: Train the CNN using a dataset of images of plant diseases.
4. Model Evaluation: Evaluate the performance of the CNN using metrics such as accuracy, precision, recall, and F1-score.

Disease Detection

1. Image Classification: Use the trained CNN to classify images of plant leaves or other parts into different disease categories.
2. Disease Identification: Identify specific types of plant diseases, such as fungal, bacterial, or viral infections.
3. Confidence Score: Provide a confidence score for each disease prediction.

User Interface

1. Web Interface: Develop a web interface for users to upload images and receive disease predictions.
2. Mobile App: Develop a mobile app for users to capture and upload images, and receive disease predictions.
3. API Integration: Integrate the plant disease detection system with other applications or services using APIs.

Database Management

1. Disease Database: Create a database of plant diseases, including images, descriptions, and management strategies.
2. Image Database: Store images of plant diseases, including metadata such as image capture date, location, and disease type.

Model Updates and Maintenance

1. Model Updates: Regularly update the CNN using new data and techniques to improve performance.
2. Model Maintenance: Monitor model performance and make adjustments as needed to ensure continued accuracy and reliability.

FEASIBILITY STUDY

FEASIBILITY STUDY

This feasibility study aims to assess the practicality and potential of using deep learning techniques for plant disease detection. The study evaluates the technical, economic, and operational feasibility of developing a plant disease detection system using deep learning.

Technical Feasibility

1. **Data Availability:** Large datasets of images of plant diseases are available, which can be used to train and test deep learning models.
2. **Computational Resources:** High-performance computing resources, such as GPUs and TPUs, are available to support the training and deployment of deep learning models.
3. **Deep Learning Frameworks:** Open-source deep learning frameworks, such as TensorFlow and PyTorch, provide a solid foundation for building and deploying deep learning models.
4. **Image Processing Techniques:** Image processing techniques, such as image resizing, normalization, and data augmentation, can be used to preprocess images and improve model performance.

Economic Feasibility

1. **Cost of Data Collection:** The cost of collecting and annotating large datasets of images of plant diseases can be significant.
2. **Cost of Computational Resources:** The cost of high-performance computing resources, such as GPUs and TPUs, can be high.
3. **Cost of Model Development:** The cost of developing and training deep learning models can be significant, requiring expertise in deep learning and image processing.
4. **Potential Return on Investment:** The potential return on investment for a plant disease detection system using deep learning is high, as it can help reduce crop losses a

Operational Feasibility

1. **User Adoption:** The adoption of a plant disease detection system using deep learning by farmers, researchers, and other stakeholders is crucial for its success.
2. **System Integration:** The system must be integrated with existing agricultural systems and workflows to ensure seamless adoption.
3. **Maintenance and Updates:** The system must be regularly maintained and updated to ensure continued accuracy and reliability.

PLATFORM

HARDWARE REQUIREMENTS

- Processor : 2.5 gigahertz (GHz) frequency or above.
- RAM : A minimum of 4 GB of RAM.
- Hard disk : A minimum of 20 GB of available space.
- Input Device : High resolution camera.
- Monitor : Minimum Resolution 1024 X 768.

SOFTWARE REQUIREMENTS

Operating System : Windows 7 and above.

Programing language : Python 2.7 and above.

Supporting libraries : TensorFlow, OpenCV, PIL,tkinter,Os,SKlearn etc.

CONCEPTS AND METHODS

CONCEPTS AND METHODS

CONCEPTS

The key concepts for "Plant Disease Detection using Deep Learning" encompass a range of techniques and methodologies aimed at effectively identifying manipulated media. **Deep Learning Concepts**

1. Convolutional Neural Networks (CNNs): CNNs are a type of deep learning model that are particularly well-suited for image classification tasks, such as plant disease detection.
2. Transfer Learning: Transfer learning is a technique that allows a pre-trained model to be fine-tuned for a specific task, such as plant disease detection.
3. Data Augmentation: Data augmentation is a technique that involves generating additional training data by applying transformations to existing images, such as rotation, flipping, and cropping.

Image Processing Concepts

1. Image Segmentation: Image segmentation is the process of dividing an image into its constituent parts, such as leaves, stems, and roots.
2. Image Enhancement: Image enhancement is the process of improving the quality of an image, such as by adjusting the brightness, contrast, and color balance.
3. Feature Extraction: Feature extraction is the process of extracting relevant features from an image, such as texture, shape, and color.

Plant Disease Detection Methods

1. Leaf Disease Detection: Leaf disease detection involves detecting diseases on leaves, such as fungal infections, bacterial infections, and viral infections.
2. Stem Disease Detection: Stem disease detection involves detecting diseases on stems, such as canker, galls, and lesions.
3. Root Disease Detection: Root disease detection involves detecting diseases on roots, such as root rot, crown rot, and root knot nematodes.

Deep Learning Architectures

1. VGG16: VGG16 is a pre-trained CNN architecture that can be fine-tuned for plant disease detection.
2. ResNet50: ResNet50 is a pre-trained CNN architecture that can be fine-tuned for plant

3. InceptionV3: InceptionV3 is a pre-trained CNN architecture that can be fine-tuned for plant disease detection.

Evaluation Metrics

1. Accuracy: Accuracy is a measure of how well a model performs, calculated as the number of correct predictions divided by the total number of predictions.
2. Precision: Precision is a measure of how well a model performs, calculated as the number of true positives divided by the sum of true positives and false positives.
3. Recall: Recall is a measure of how well a model performs, calculated as the number of true positives divided by the sum of true positives and false negatives.
4. F1-Score: F1-score is a measure of how well a model performs, calculated as the harmonic mean of precision and recall.

ALGORITHMS

Object Detection Algorithms

1. YOLO (You Only Look Once): A real-time object detection algorithm that can be used for plant disease detection.
2. SSD (Single Shot Detector): A real-time object detection algorithm that can be used for plant disease detection.
3. Faster R-CNN (Region-based Convolutional Neural Networks): A object detection algorithm that can be used for plant disease detection.

Image Segmentation Algorithms

1. U-Net: A CNN architecture that can be used for image segmentation tasks, such as plant disease detection.
2. SegNet: A CNN architecture that can be used for image segmentation tasks, such as plant disease detection.
3. FCN (Fully Convolutional Networks): A CNN architecture that can be used for image segmentation tasks, such as plant disease detection.

Other Algorithms

1. SVM (Support Vector Machines): A machine learning algorithm that can be used for plant disease detection.
2. Random Forest: A machine learning algorithm that can be used for plant disease detection.
3. Gradient Boosting: A machine learning algorithm that can be used for plant disease detection.

METHODOLOGY

The project "Plant Disease Detection using Deep Learning" typically follows common methodologies in deep learning and computer vision research.

Data Collection

1. Collect images of plant diseases: Collect a large dataset of images of plant diseases, including images of healthy plants and plants with various diseases.
2. Label the images: Label the images with the corresponding disease class, such as "healthy", "disease1", "disease2", etc.

Data Preprocessing

1. Resize images: Resize the images to a uniform size to reduce computational requirements.
2. Normalize images: Normalize the images to have similar pixel values, which can improve model performance.
3. Data augmentation: Apply data augmentation techniques, such as rotation, flipping, and cropping, to increase the size of the dataset.

Deep Learning Model Selection

1. Choose a deep learning architecture: Choose a deep learning architecture, such as a convolutional neural network (CNN), that is suitable for image classification tasks.
2. Select a pre-trained model: Select a pre-trained model, such as VGG16 or ResNet50, that can be fine-tuned for plant disease detection.

Model Training

1. Split the dataset: Split the dataset into training, validation, and testing sets.
2. Train the model: Train the model using the training set and validate its performance using the validation set.
3. Fine-tune the model: Fine-tune the model by adjusting its hyperparameters and architecture to improve its performance.

Model Evaluation

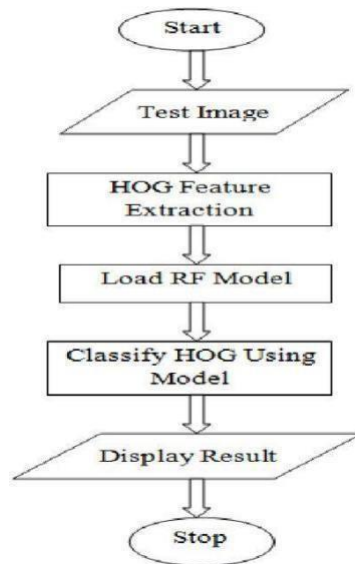
1. Evaluate the model: Evaluate the model's performance using metrics, such as accuracy, precision, recall, and F1-score.
2. Compare with other models: Compare the model's performance with other models, such as machine learning models or other deep learning models.

Deployment

1. Deploy the model: Deploy the model in a suitable platform, such as a web application or a mobile app.
2. Integrate with other systems: Integrate the model with other systems, such as a database or a decision support system.

UML DIAGRAMS

UML DIAGRAM



USE CASE DIAGRAM:

Use case diagram is a graphic depiction of the interactions among the of a system. Use cases will specify the expected behavior, and the exact method of making it happen. Use cases once specified can be denoted both textual and visual representation.

Use case diagrams are used to specify:

□

Requirements (external) required usages of a system under design or analysis - to capture what the system is supposed to do.

The functionality offered by a subject – what the system can do. Requirements the specified subject poses on its environment - by defining how environment should interact with the subject so that it will be able to perform its services.

CLASS DIAGRAM

It focuses on the static structure of a system by showing its classes, attributes, operations, and relationships.

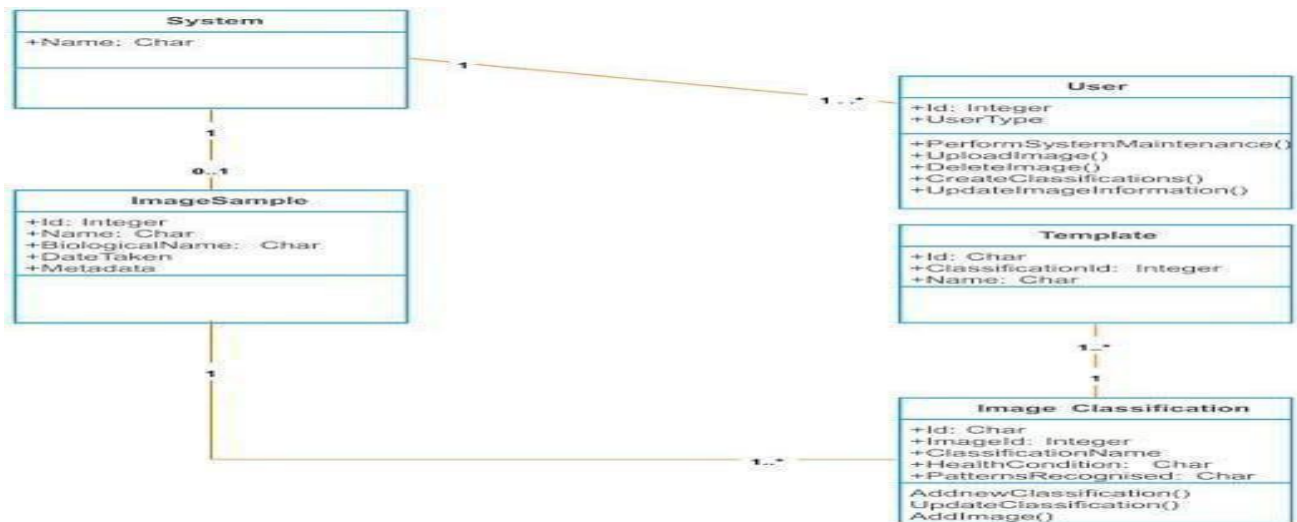


Fig : Class diagram

ACTIVITY DIAGRAM:

Represents user interactions with the system. The process flows in the system are captured in the activity diagram. Similar to a state diagram, an activity diagram also consists of activities, actions, transitions, initial and final states, and guard conditions.

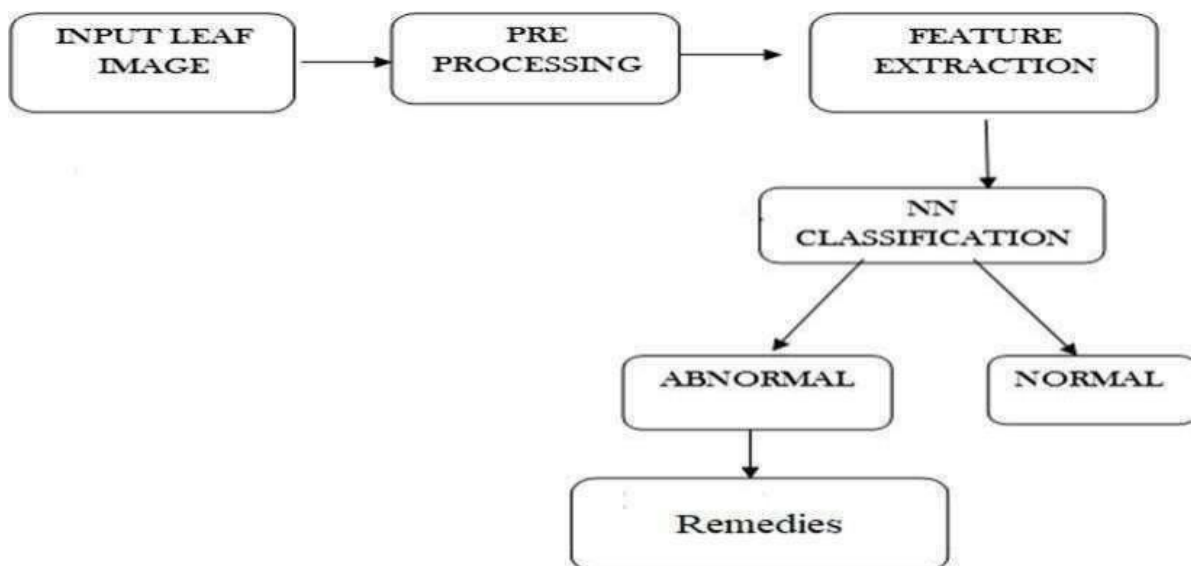


Fig : Activity diagram

SEQUENCE DIAGRAM:

Sequence diagrams are sometimes called event diagrams or event scenarios. A sequence diagram shows as parallel vertical lines (lifelines), different processes or objects that live simultaneously and as horizontal arrows, the messages exchanged between them in the order in which they occur.

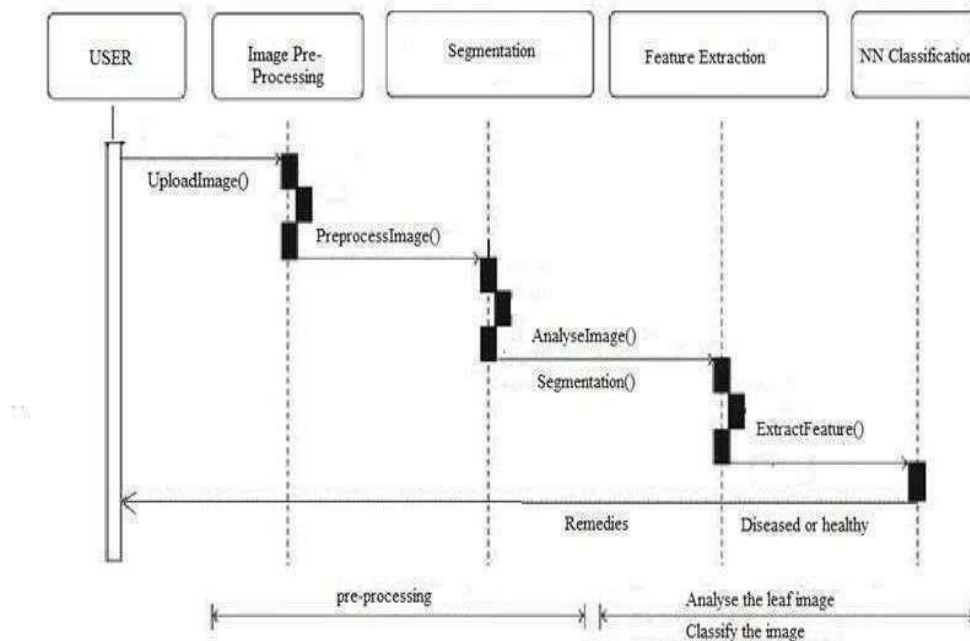


Fig : Sequence diagram

Usage scenarios: A usage scenario is a description of a potential way your system is used. The logic of a usage scenario may be part of a use case, perhaps an alternate course. It may also be one entire pass through a use case, such as the logic described by the basic course of action or a portion of the basic course of action, plus one or more alternate scenarios. The logic of a usage scenario may also be a pass through the logic contained in several use cases.

The logic of methods: Sequence diagrams can be used to explore the logic of a complex operation, function, or procedure. One way to think of sequence diagrams, particularly highly detailed diagrams, is as visual object code

The logic of services: A service is effectively a high-level method, often one that can be invoked by a wide variety of clients. This includes web-services as well as business transactions implemented by a variety of technologies.

COLLABORATION DIAGRAM:

A collaboration diagram huddles the interactions among different objects. The interactions are detailed as numbered interactions, aiding in tracing the sequence of interactions. Each object's potential interactions with other objects are identified using the collaboration diagram.

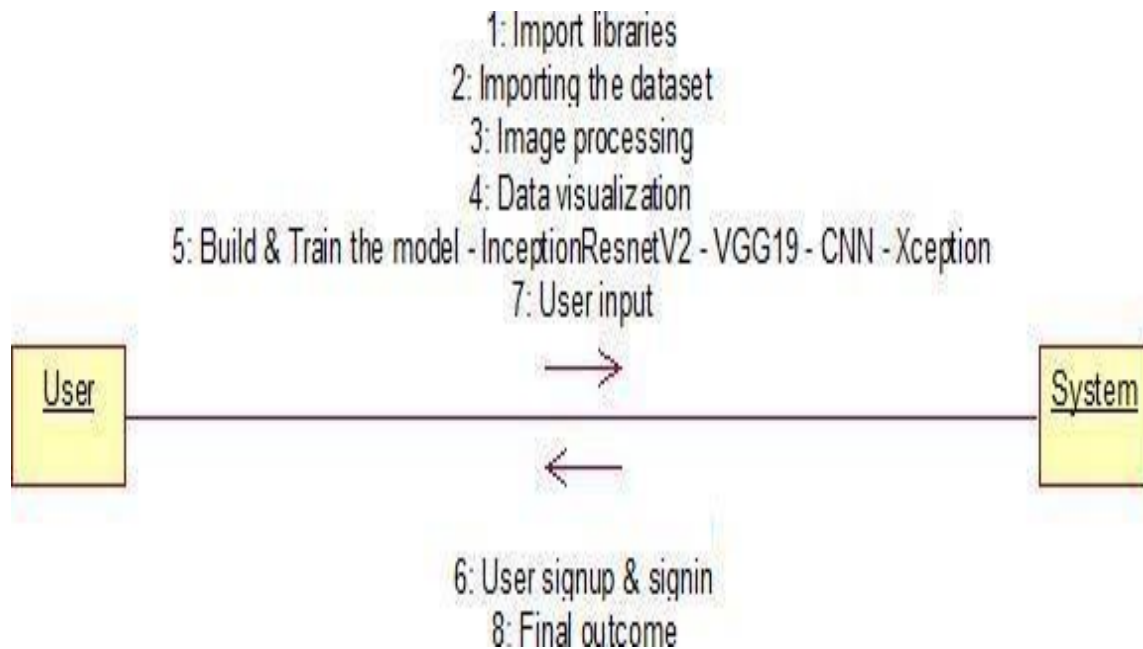


fig: Collaboration diagram

COMPONENT DIAGRAM:

The component diagram represents the high-level parts that make up the system. This diagram depicts, at a high level, what components form part of the system and how they are interrelated. A component diagram depicts the components culled after the system has undergone the development or construction phase.

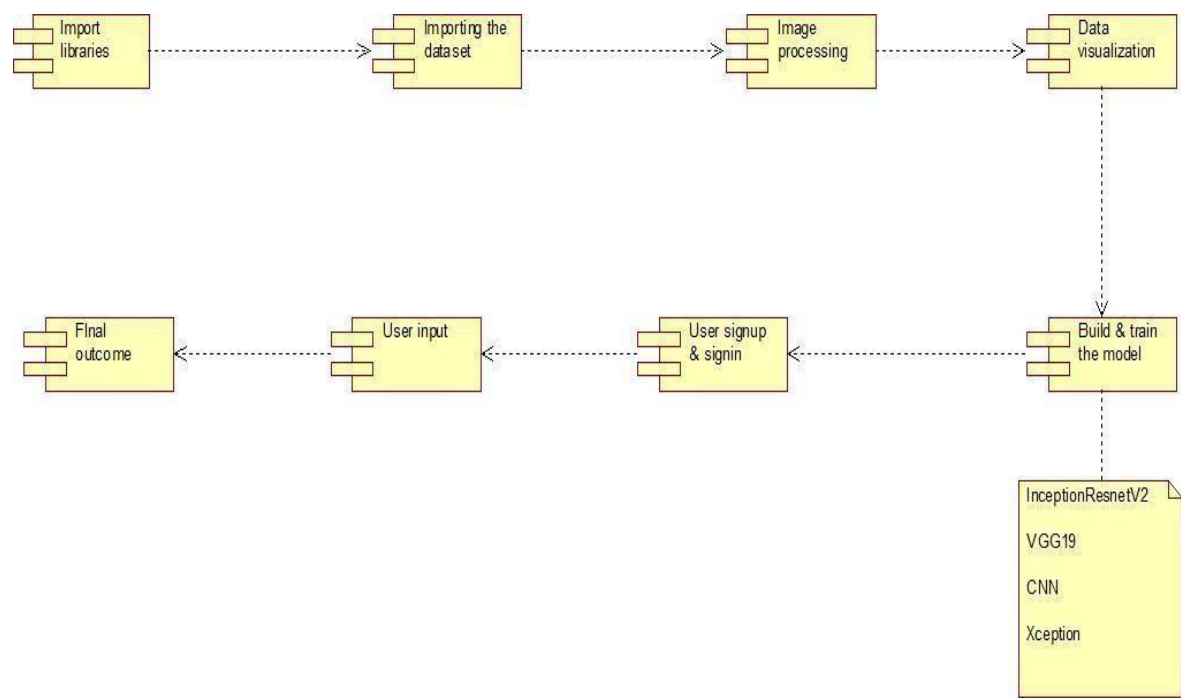


fig 4.6: Component diagram

SOFTWARE ENVIRONMENT

SOFTWARE ENVIRONMENT

WHAT IS PYTHON:

Python is presently the maximum extensively used multi-purpose, high-stage programming language. Python packages typically are smaller than different programming languages like Java. Programmers have to type relatively less, and the language's indentation requirement makes them readable all the time. Python language is being utilized by nearly all tech-massive businesses like Google, Amazon, Facebook, Instagram, Dropbox, Uber, etc. The enormous electricity of Python is a large series of fashionable libraries that may be applied for the following:

1. Machine Learning
2. GUI Applications
3. Web frameworks like Django
4. Image processing
5. Test frameworks
6. Multimedia

Advantages of Python:

- 1. Extensive Libraries:** Python comes with a comprehensive library containing code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. Thus, there is no need to manually write the complete code for those tasks.
- 2. Extensible:** Python has the capability to be extended to other languages. Portions of code can be written in languages like C++ or C, which is beneficial, especially for projects.
- 3. Embeddable:** In addition to its extensibility, Python is also embeddable. Python code can be inserted into the source code of a different language, such as C++. This enables the addition of scripting

1. **Improved Productivity:** The simplicity of the language and its extensive libraries make programmers more productive compared to languages like Java and C++. Python requires writing less code to accomplish more tasks.
2. **Simple and Easy:** Unlike Java where creating a class to print 'Hello World' is necessary, in Python, a simple print statement suffices. Python is easy to learn, understand, and code which is why transitioning to other more verbose languages like Java can be challenging once accustomed to Python.

Disadvantages of Python:

While Python offers several advantages, its downsides should be considered when choosing it over another language:

1. **Speed Limitations:** Python executes code line by line since it is interpreted, leading to slower execution. However, unless high speed is crucial for the project, Python's benefits are usually sufficient to offset its speed limitations. Weak in Mobile Computing and Browsers While excellent as a server-side language, Python is rarely used on the client-side and is not typically employed in developing smartphone-based applications due to security concerns.
2. **Design Restrictions:** Python is dynamically-typed, eliminating the need to declare variable types when writing code. It utilizes duck-typing which although convenient during coding can result in runtime errors.
3. **Underdeveloped Database Access Layers:** Python's database access layers are less developed compared to technologies like JDBC and ODBC, limiting its application in large enterprises.

WHAT IS DEEP LEARNING :

Deep learning is a subset of machine learning that involves the use of artificial neural networks to analyze and interpret data. It is a type of machine learning that is inspired by the structure and function of the brain, and is designed to learn complex patterns and relationships in data.

How Deep Learning Works

Deep learning works by using a series of layers to analyze data. Each layer is made up of a set of nodes or "neurons" that process the input data and pass it on to the next layer. The layers are designed to learn increasingly complex patterns and relationships in the data, allowing the model to make accurate predictions or classifications.

Advantages of Deep Learning

1. Improved Accuracy: Deep learning models can achieve state-of-the-art performance on a wide range of tasks, including image and speech recognition, natural language processing, and more.
2. Ability to Learn Complex Patterns: Deep learning models can learn complex patterns and relationships in data, making them well-suited for tasks such as image and speech recognition.
3. Flexibility: Deep learning models can be used for a wide range of tasks, including classification, regression, and clustering.
4. Ability to Handle Large Amounts of Data: Deep learning models can handle large amounts of data, making them well-suited for tasks such as image and speech recognition.
5. Improved Performance on Unstructured Data: Deep learning models can perform well on unstructured data, such as images and audio, which can be challenging for traditional machine learning models.

What is Anaconda for Python?

Anaconda for Python is a software package that facilitates the management of Python environments and packages. It enables users to create environments with various Python versions and specific package versions tailored to their projects. Additionally, Anaconda simplifies package installation, removal, and upgrades within these environments, streamlining the development process. Anaconda is also equipped with deployment capabilities, allowing users to deploy projects effortlessly with just a few clicks. This feature makes it particularly appealing to beginners looking to learn Python programming.

How to install Anaconda for Python?

When installing Anaconda, users can visit the Anaconda Documentation website and download the installer suitable for their operating system. Upon downloading the installer, they can initiate the installation process by double-clicking on it. The installation prompts guide users through the process, prompting them to agree to terms and conditions and choose options such as adding Anaconda to the system's PATH environment variable.

Selecting "yes" when asked to add Anaconda to the PATH environment variable ensures that the system can locate Anaconda files efficiently. Furthermore, users are typically prompted to enable Anaconda as the default Python interpreter, which is recommended for seamless integration.

Python Anaconda Installation:

As of the latest available version (2019.10), users can follow these steps to install Anaconda on their machines. Following these instructions will provide users with access to Anaconda's comprehensive suite of tools and functionalities, empowering them to manage Python environments and packages effectively for their projects.

Download the Correct version into the system:

Step 1: Go to browser and download Anaconda for Windows, Mac, or Linux

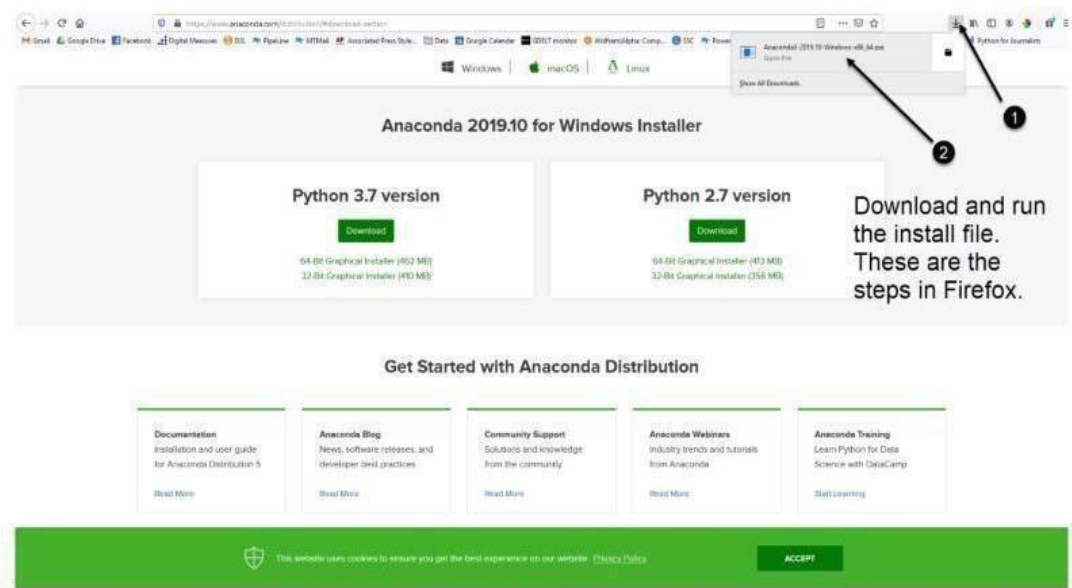


Fig 5.1 : Screen for install anaconda python using google chrome

Step 2: Choose where you want to install it. Here, you can see the available space and how much you need

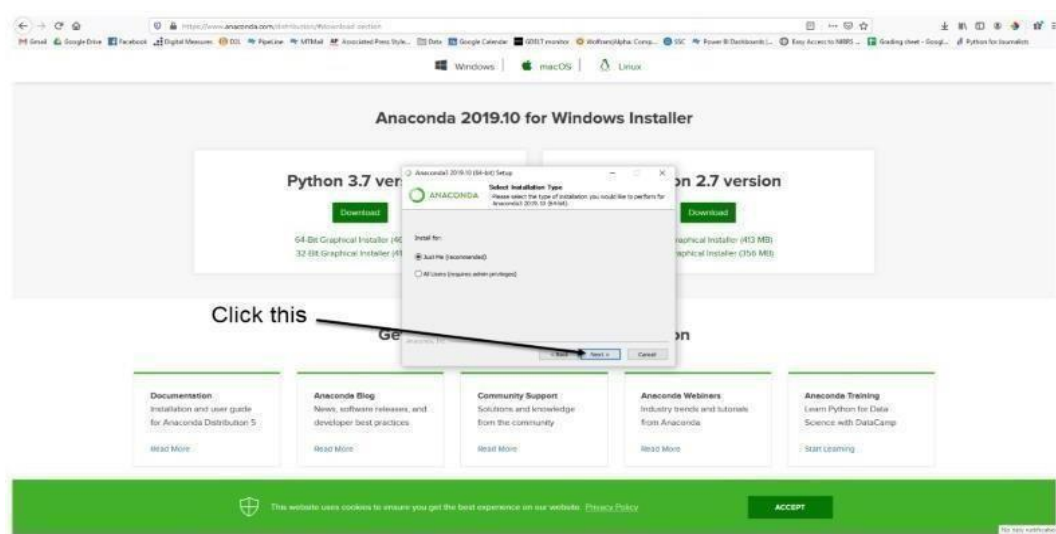


Fig 5.2: Screen for download the latest version

Step 3: Now, you'll get some advanced options. You can add Anaconda to your system's PATH environment variable, and register it as the primary system Python 3.7. If you add it to PATH, it will be found before any other installation. Click on 'Install'

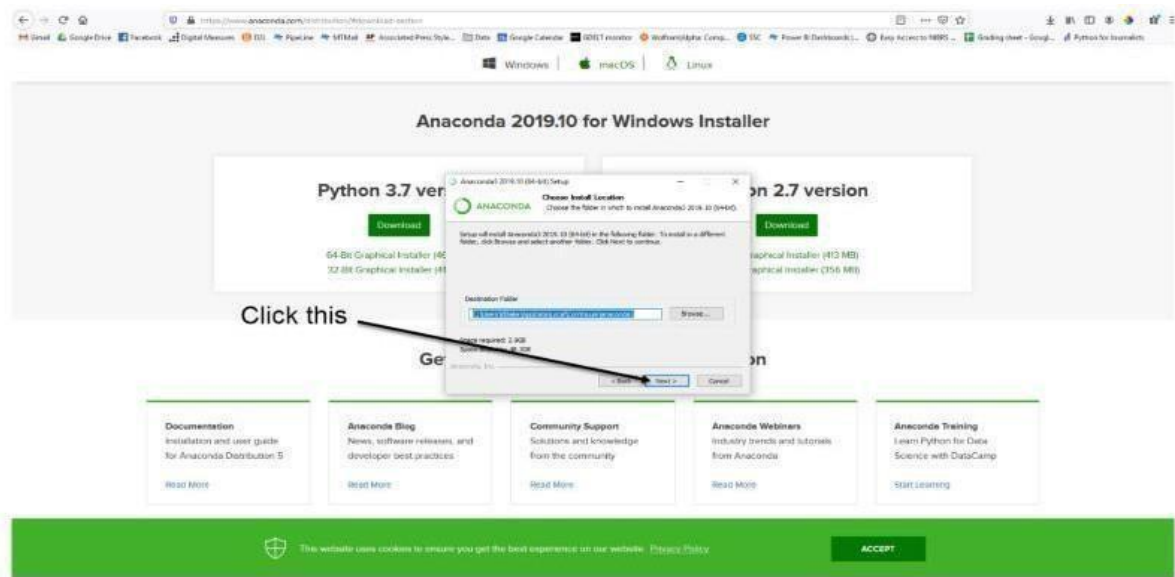


Fig 5.3: Screen for Setting path to download

Step 4: It will unpack some packages and extract some files on your machine. This will take a few minutes

Step 5: This screen will inform you about PyCharm. Click Next.



Fig 5.4: Screen for infor2m8 of Pycharm

Step 6: The installation is complete. You can choose to get more information about Anaconda cloud and how to get started with Anaconda. Click Finish.

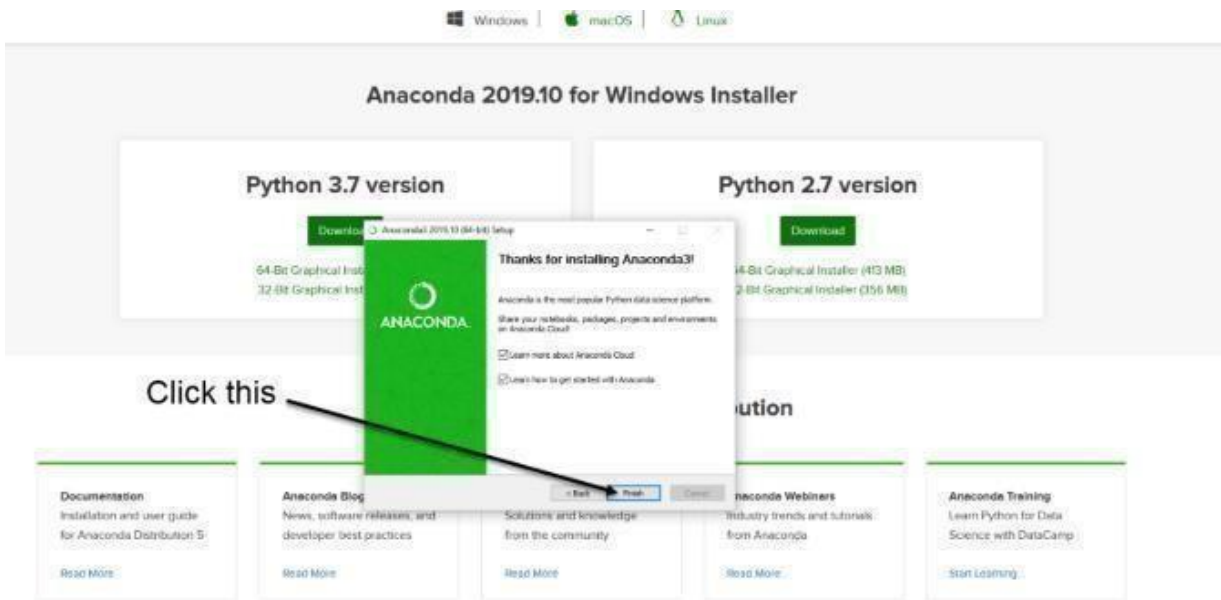


Fig 5.5: Screen for installation complete

Step 7: If you search for Anaconda now, you will see the following options

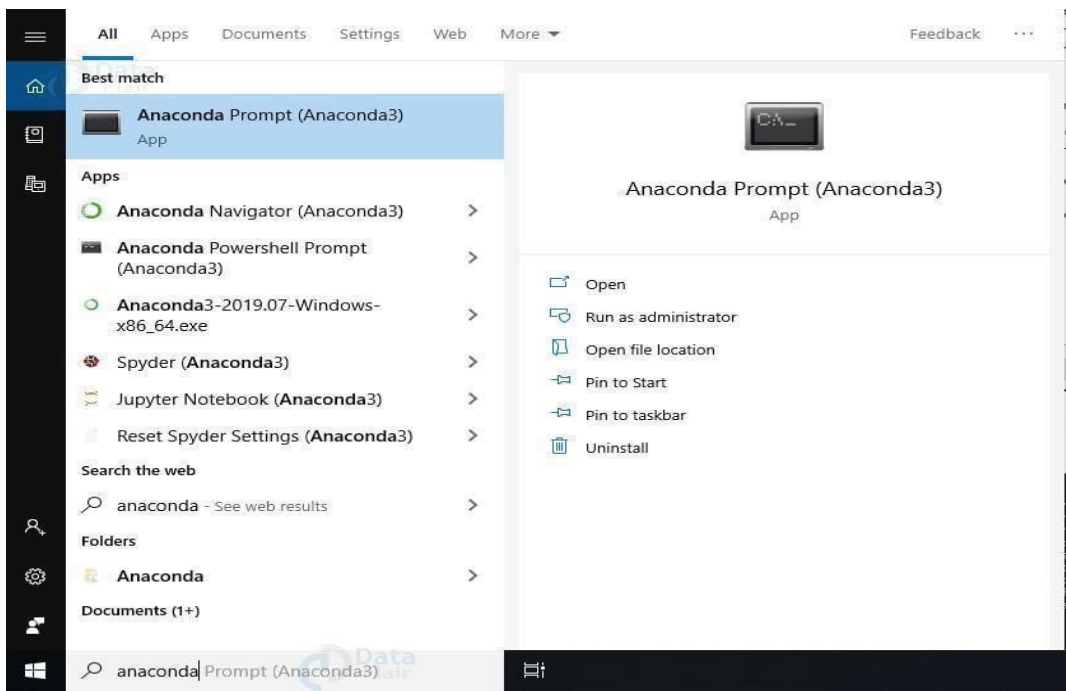


Fig 5.6: Screen for anaconda prompt

SYSTEM TESTING

SYSTEM TESTING

Software testing is an investigation conducted to provide stakeholders with information about the quality of the product or service under test. Software testing can also provide an objective, independent view of the software to allow the business to appreciate and understand the risks of software implementation. There are various types of testing, explained in the following sections.

Testing Objective

Testing is a process of executing a program with the intent of finding an error.

A good test case is one that has a high probability of finding a yet undiscovered error.

A successful test one that uncovers an as –yet undiscovered error.

The above objectives imply a dramatic change in viewpoint. They move counter to the commonly held view that a successful test is one in which no errors are found. Testing cannot show the absence of defects; it can only show that software errors are present.

The different levels of testing that are to be conducted are:

- Unit Testing
- Integration Testing
- System Testing
- Code Testing
- Program Testing

TEST STRATEGY

Test Objectives

1. **Verify Accuracy:** Verify that the deep learning model can accurately detect plant diseases from images.
2. **Evaluate Performance:** Evaluate the performance of the model on different types of plant diseases and images.
3. **Identify False Positives and Negatives:** Identify false positives and negatives to improve the model's performance.

Test Scope

1. **Image Dataset:** The test will use a dataset of images of plant diseases, including images of healthy plants and plants with various diseases.
2. **Deep Learning Model:** The test will use a deep learning model trained on the image dataset to detect plant diseases.
3. **Test Environment:** The test will be conducted in a controlled environment with a specific set of hardware and software configurations.

TEST CASES

Table : Unit Testing Table crop recommendation

Test case	Crop Recommendation
Precondition	Application must be launched and select CROP from our services
Description	Suitable crop will be recommended
Output	Based on the values provided in the screen, the suitable crop will be recommended
Status	Tested and worked for multiple values of Nitrogen, Potassium, Phosphorus, Ph Level, Rainfall, State and City

Table : Unit Testing Table for fertilizer recommendation

Test case	Fertilizer Recommendation
Precondition	Select FERTILIZER from our services
Description	Suitable fertilizer will be recommended for the crop
Output	Based on the values provided in the screen, the suitable fertilizer will be recommended
Status	Tested and worked for multiple values of Nitrogen, Potassium, Phosphorus, Crop.

Test Cases

Test case	Crop disease detection
Precondition	Select CROP DISEASE DETECTION from our services
Description	Disease name and remedies will be recommended
Output	Based on the uploaded image, if the disease presents, the disease name and remedies will be recommended. Else, only Healthy message will be displayed.
Status	Tested and worked for uploading different types and sizes of images and if the remedies are displayed successfully

Table : Unit Testing Table for crop disease detection

IMPLEMENTATION

IMPLEMENTATION

SAMPLE CODE:

```
# Importing essential libraries and modules

from flask import Flask, render_template, request, Markup
import numpy as np
import pandas as pd
from utils.disease import disease_dic from
utils.fertilizer import fertilizer_dic import
requests
import config import
pickle import io import
torch
from torchvision
import transforms from PIL import Image
from utils.model import ResNet9 #
=====
=====
#          _____ LOADING THE TRAINED MODELS _____
          _____

# Loading plant disease classification model

disease_classes = ['Apple.....Apple_scab',
'Apple Black_rot',
'Apple___Cedar_apple_rust',
```

'Apple _ healthy', 'Blueberry _ healthy',
 'Cherry_(including_sour) _ Powdery_mildew',
 'Cherry_(including_sour) _ healthy',
 'Corn_(maize) _ Cercospora_leaf_spot Gray_leaf_spot', 'Corn_(maize) _
 Common_rust',
 'Corn_(maize) __Northern_Leaf_Blight',
 'Corn_(maize) _ healthy',
 'Grape _ Black_rot',
 'Grape _ Esca(Black_Measles)',
 'Grape _ Leaf_blight(Isariopsis_Leaf_Spot)', 'Grape_healthy',
 'Orange _ Haunglongbing(Citrus_greening)',
 'Peach_Bacterial_spot',
 'Peach_healthy', 'Pepper,bell _ Bacterial_spot',
 'Pepper,bell __ healthy', 'Potato _ Early_blight',
 'Potat_Late_blight', 'Potato_healthy',
 'Raspberry_healthy', 'Soybean __ healthy',
 'Squash_Powdery_mildew',
 'Strawberry_Leaf_scorch', 'Strawberry_healthy',
 'Tomato_Bacterial_spot', 'Tomato _ Early_blight',
 'Tomato_Late_blight', 'Tomato _ Leaf_Mold',

```

'Tomato Septoria_leaf_spot',
'Tomato_Spider_mites Two
spotted_spider_mite', Tomato Target_Spot',
'Tomato_Tomato_Yellow_Leaf_Cur_1_Virus'
,'Tomato Tomato_mosaic_virus', 'Tomato healthy']

```

```

disease_model_path = 'models/plant_disease_model.pth' disease_model =
ResNet9(3, len(disease_classes))
disease_model.load_state_dict(torch.load(
disease_model_path, map_location=torch.device('cpu')))) disease_model.eval()

```

```

# Loading crop recommendation model

```

```

crop_recommendation_model_path = 'models/RandomForest.pkl'
crop_recommendation_model = pickle.load(
open(crop_recommendation_model_path, 'rb'))

```

```

# Custom functions for calculations

```

```

def weather_fetch(city_name): """
Fetch and returns the temperature and humidity of a city
:params: city_name
:return: temperature,

humidity """
api_key = config.weather_api_key
base_url

complete_url = base_url + "appid=" + api_key + "&q=" + city_name response =
requests.get(complete_url)
x = response.json()

```

```

if x["cod"] != "404": y =
x["main"]
temperature = round((y["temp"] - 273.15), 2) humidity =
y["humidity"]
return temperature, humidity else:
return None

def predict_image(img, model=disease_model): """
Transforms image to tensor and predicts disease label
:params: image
:return: prediction (string) """
transform = transforms.Compose([
transforms.Resize(256), transforms.ToTensor(),
])
image = Image.open(io.BytesIO(img)) img_t = transform(image)img_u = torch.unsqueeze(img_t, 0)

# Get predictions from model

yb = model(img_u)
# Pick index with highest probability _, preds
= torch.max(yb, dim=1)
prediction = disease_classes[preds[0].item()] # Retrieve the
class label
return prediction

#
=====
=====
== # ----- FLASK APP -----
app = Flask(__name__)

```

```
# render home page

@ app.route('/') def home(): title = 'Plant
Disease -Home' return
render_template('index.html' , title=title)

# render crop recommendation form page

@ app.route('/crop-recommend') def
crop_recommend():
title = ' Crop Recommendation'
_____
return render_template('crop.html', title=title
```



```

# render fertilizer recommendation form page

    @ app.route('/fertilizer')
def fertilizer_recommendation():
    title = 'Harvestify - Fertilizer Suggestion'

    return render_template('fertilizer.html', title=title)

# render disease prediction input page

# RENDER PREDICTION PAGES

# render crop recommendation result page


@ app.route('/crop-predict', methods=['POST']) def
crop_prediction():
    title = 'Harvestify - Crop Recommendation' if
    request.method == 'POST':N =
    int(request.form['nitrogen'])

    P = int(request.form['phosphorous']) K =
    int(request.form['pottasium']) ph =
    float(request.form['ph'])
    rainfall = float(request.form['rainfall'])

    # state = request.form.get("stt") city =
    request.form.get("city")
    if weather_fetch(city) != None:

    temperature, humidity = weather_fetch(city)
    data = np.array([[N, P, K, temperature, humidity, ph, rainfall]])

    my_prediction = crop_recommendation_model.predict(data)

    final_prediction = my_prediction[0]

```

```

return render_template('crop-result.html',
prediction=final_prediction, title=title)
else:

return render_template('try_again.html', title=title)

# render fertilizer recommendation result page

@app.route('/fertilizer-predict', methods=['POST']) def
fert_recommend():
title = 'Harvestify - Fertilizer Suggestion'

crop_name = str(request.form['cropname']) N =
int(request.form['nitrogen'])
P = int(request.form['phosphorous']) K =
int(request.form['pottasium'])
# ph = float(request.form['ph'])

df = pd.read_csv('Data/fertilizer.csv')

nr = df[df['Crop'] == crop_name]['N'].iloc[0]
pr = df[df['Crop'] == crop_name]['P'].iloc[0]

```

```

kr = df[df['Crop'] == crop_name]['K'].iloc[0]

n = nr - N

p = pr - P k= kr - K

temp = {abs(n): "N", abs(p): "P", abs(k): "K"}

max_value = temp[max(temp.keys())]

if max_value == "N": if n < 0:
key = 'NHigh' else:
key = "Nlow"

elif max_value == "P": if p < 0:
key = 'PHigh' else:
key = "Plow"

else:
if k < 0:
key = 'KHigh' else:
key = "Klow"

response = Markup(str(fertilizer_dic[key]))

return render_template('fertilizer-result.html', recommendation=response, title=title)

```

```

# render disease prediction result page @app.route('/disease-predict',
methods=['GET', 'POST']) def disease_prediction():
title = 'Plant - Disease Detection'

if request.method == 'POST': if
'file' not in request.files:

Return

redirect(request.url) file =
request.files.get('file') if not file:
return render_template('disease.html', title=title) try:
img = file.read()

prediction = predict_image(img)

prediction = Markup(str(disease_dic[prediction]))
return render_template('disease-result.html', prediction=prediction, title=title) except: pass
return render_template('disease.html', title=title)

if __name__ == '__main__':
app.run(debug=False)

```

RESULTS

SCREENSHOTS

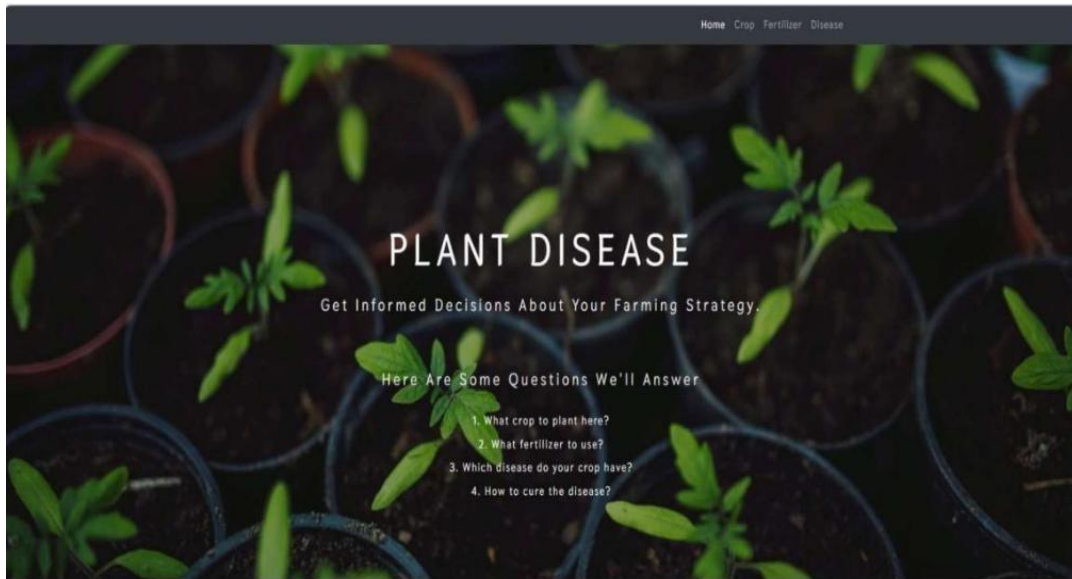


Fig : Home screen

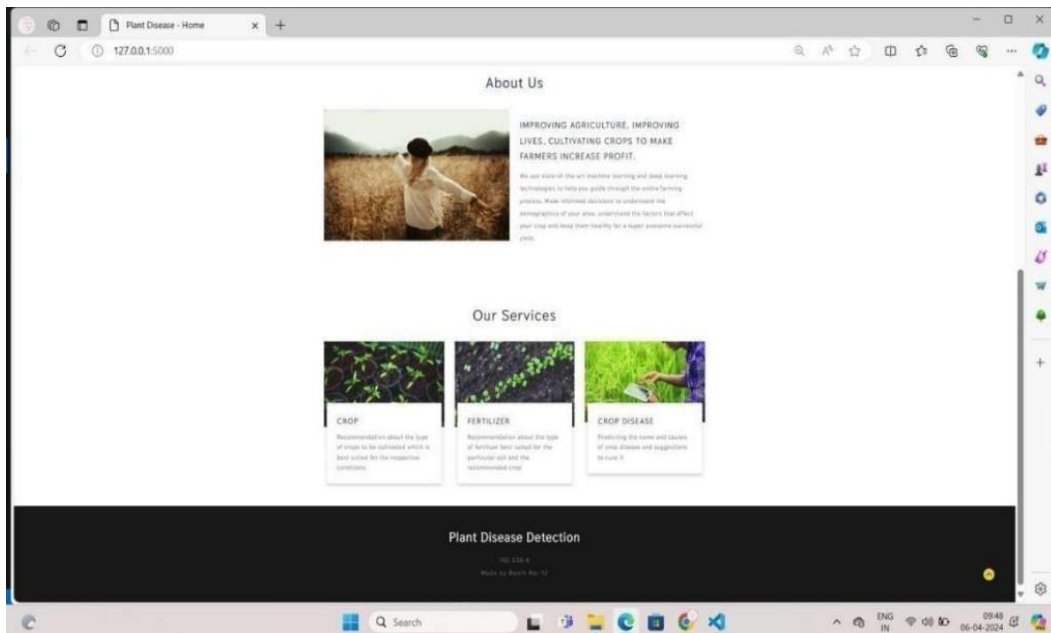


Fig : Website Services

Find out the most suitable crop to grow in your farm

Nitrogen
Enter the value (example:50)

Phosphorous
Enter the value (example:50)

Pottasium
Enter the value (example:50)

pH level
Enter the value

Rainfall (in mm)
Enter the value

State
Select State

City

Fig : Screen for Login In page

Find out the most suitable crop to grow in your farm

Nitrogen
Enter the value (example:50)

Phosphorous
Enter the value (example:50)

Pottasium
Enter the value (example:50)

pH level
Enter the value

Rainfall (in mm)
Enter the value

State
Select State

City

Predict

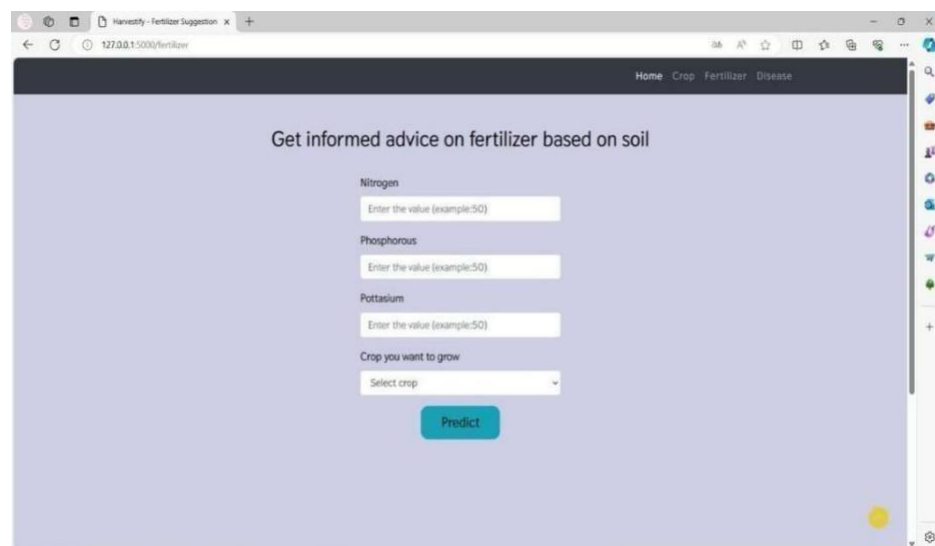
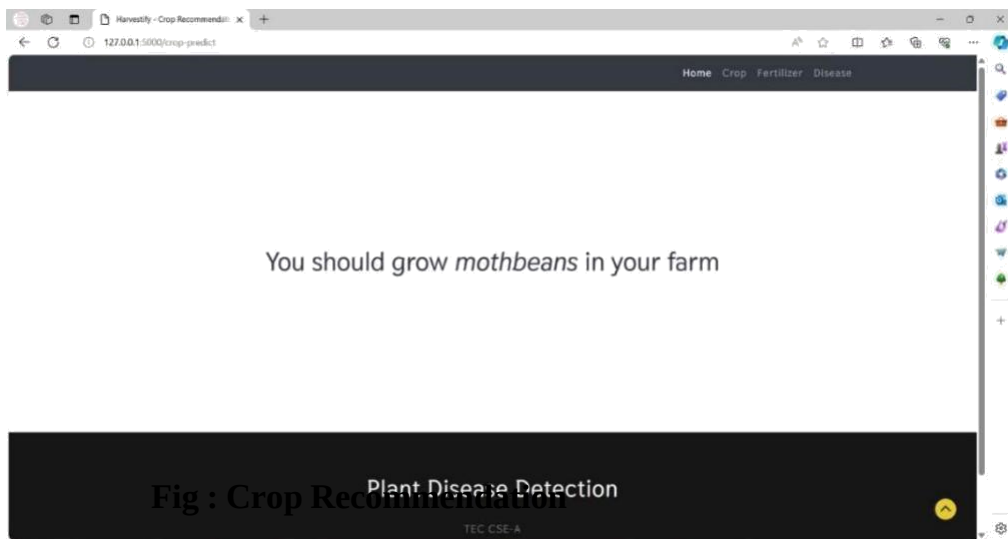


Fig : Finding Suitable Fertilizer

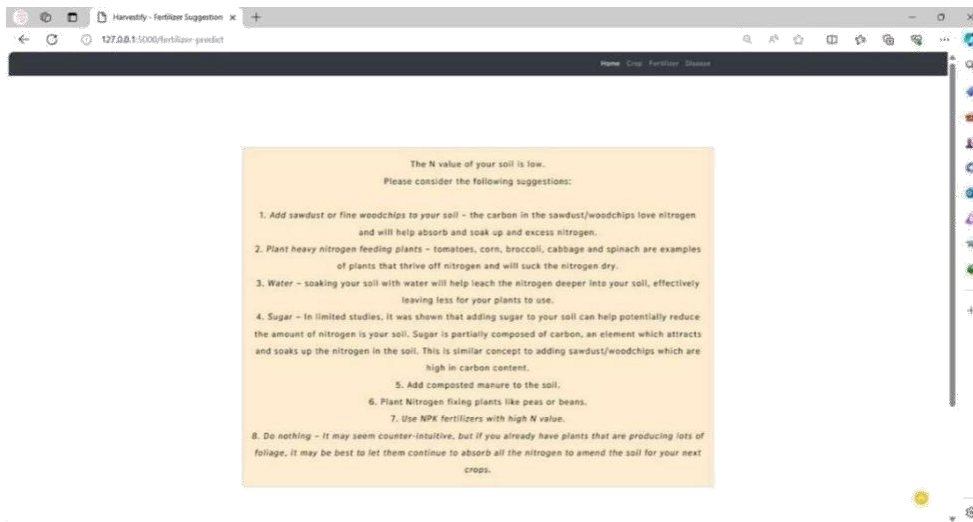


Fig: Fertilizer Recommendation

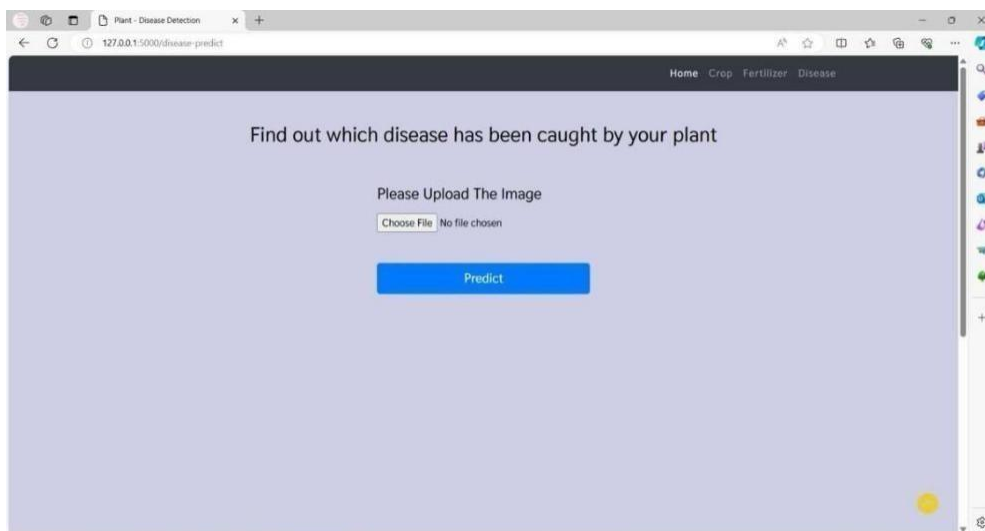


Fig : Uploading leaf image screen.

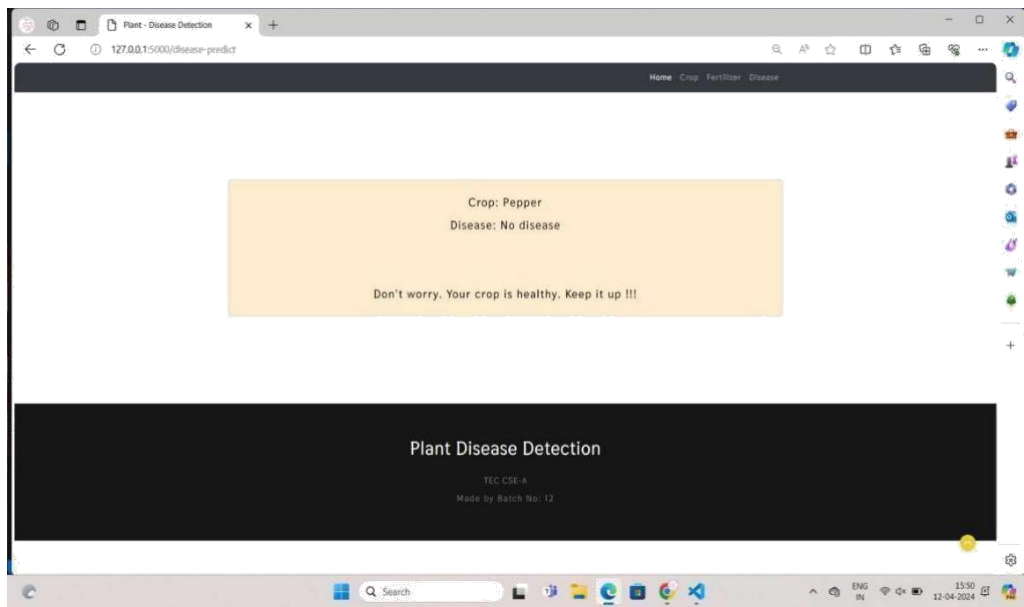


Fig: Output Screen for healthy crop

1.1 RESULTS AND COMPARISON

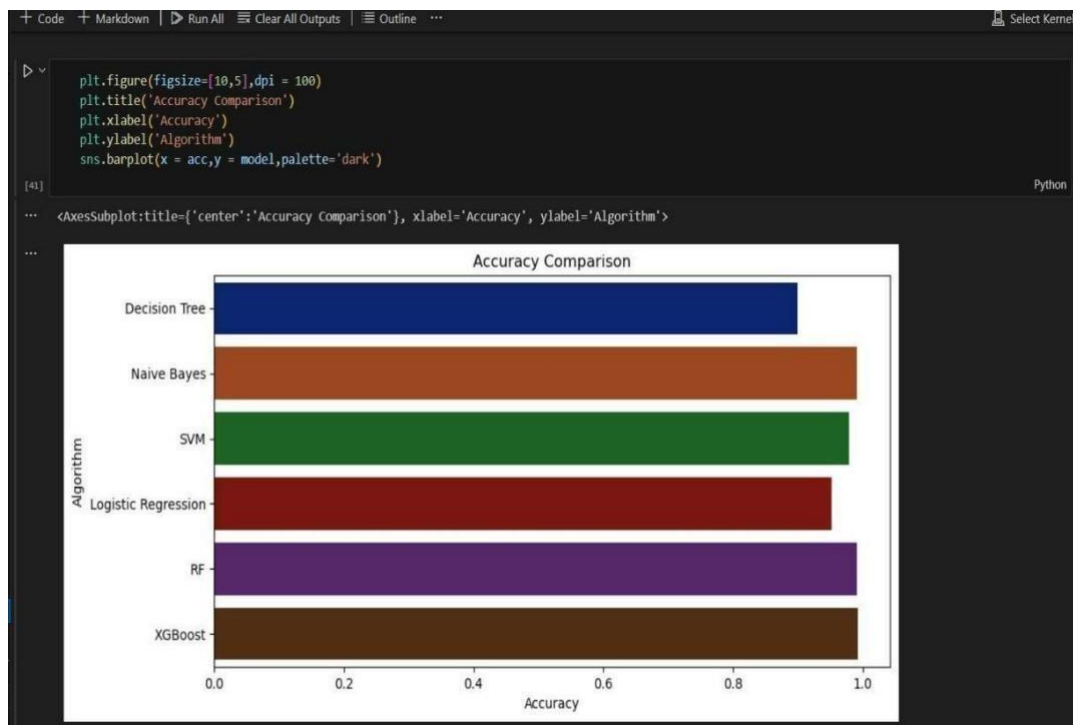
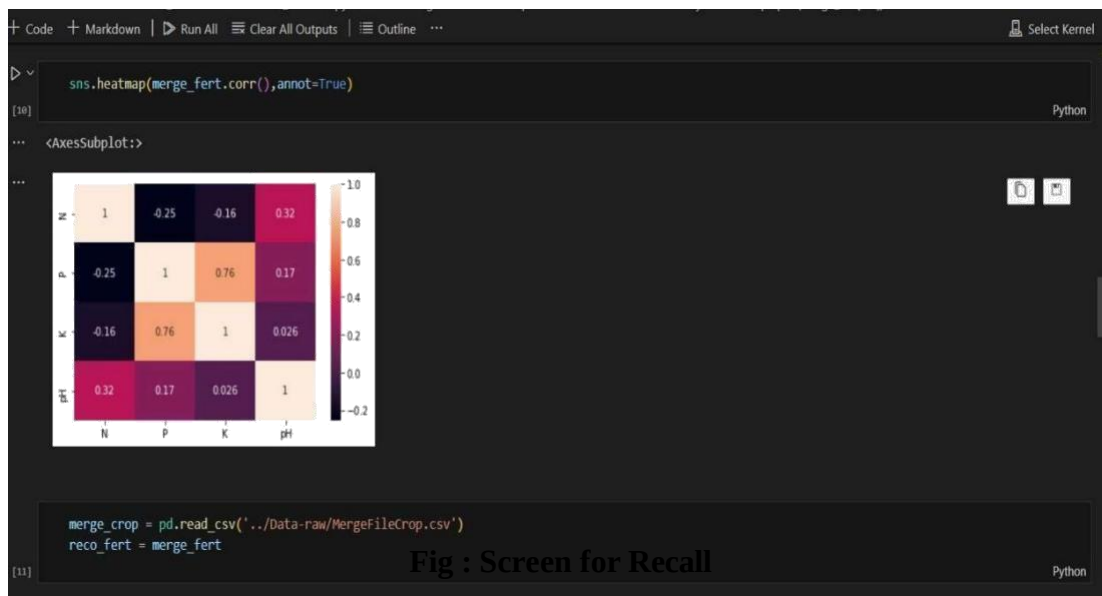


Fig: Screen for Accuracy Comparison



FUTURE ENHANCEMENTS

FUTURE ENHANCEMENTS

- To improve recognition rate of final classification process hybrid algorithms like Artificial Neural Network, Bayes classifier, Fuzzy Logic can also be used.
-
- Mobile application can be developed which is handy and easy to use.
- An extension of this work will focus on automatically estimating the severity of the detected disease.
- As future enhancement of the project is to develop the open multimedia. (Audio/Video) about the diseases and their solution automatically once the disease is detected.
- In contrast to traditional image processing techniques, which intervene in the detection of plant and pest diseases in stages, deep learning-based methods integrate them into an end-to-end extraction of characteristics.
- This has a lot of potential. Despite rapid advances in plant and pest disease detection technology, it has moved from academic research to agricultural application. The mature application still requires a great deal of work, and several issues must be resolved before it can be used in the real world.

CONCLUSION

CONCLUSION

There are number of ways by which we can detect disease of plants and suggest remedies for them. Each has some pros as well as limitations. On one hand visual analysis is least expensive and simple method, it is not as efficient and reliable. Image processing is a technique which is most spoken for very high accuracy and least time consumption are major advantages offered. The applications of K-means clustering, and Neural Networks (NNs) have been formulated for clustering and classification of diseases that effect on plant leaves. Recognizing the disease accurately and efficiently is mainly the purpose of the proposed approach.

The experimental results indicate that the proposed approach is a valuable approach, which can significantly support an accurate detection of leaf diseases in a little computational effort. Alongside the supply of cultivation tools, the farmers also need access to accurate information that they can use for efficient crop management and there is no better way than providing them a service that they can use through the software.

The first convolutional layer filters the input image with 32 kernels of size 3x3. After max-pooling is applied, the output is given as an input for the second convolutional layer with 64 kernels of size 4x4. The last convolutional layer has 128 kernels of size 1x1 followed by a fully connected layer of 512 neurons. The output of this layer is given to SoftMax function which produces a probability distribution of the four output classes.

BIBLIOGRAPHY

BIBLIOGRAPHY

1. The Basepaper is taken from <https://ieeexplore.ieee.org/document/9399342>
2. **K. Elangoran, S. Nalini, 2011** “Detection and classification of leaf diseases using K- means-based segmentation and neural-networks-based classification.” Inform. Technol. J., 10: 267-275. DOI: 10.3923/itj.2011.267.275.
3. **Sandesh Raut, Karthik Ingale**, “Review on leaf disease detection using Image Processing techniques.”
4. “A Survey on Methods of Plant Disease Detection” **Sagar Patil, Anjali Chandavale**
5. **T. RUMPF, A-K Mahlein, U sleiner, H. W. Dehne.** "Texture analysis for diagnosing paddy disease." In International Conference on Electrical Engineering and Informatics, 2009. ICEEI'09., vol. 1, pp. 23-27. IEEE, 2009.
6. “Plant disease detection and classification using image processing and artificial neural networks.” **Mr. Sanjay Mirchandani1,**
7. “Detection and Classification of Plant LeaMihir Pendse2, Prathamesh Rane3, Ashwini Vedula4f Diseases Using Image Processing Techniques:” **Savita N. Ghaiwat, Parul Arora**
8. “Image Processing Based Leaf Rot Disease, Detection of Betel Vine (Piper Betel.)” **Amar Kumar Deya*, Manisha Sharmaa, M.R. Meshramb**
9. “Advances in image processing for plant disease detection” **Jayamala k Patil, Rajkumar**
10. **S Arivazhagan, R Newlin shebiah, S Ananthi, S Vishnu varthini** “Detection of unhealthy region of plant leaves and classification of plant leaf diseases using texture features.”

PLAGARISM REPORT

PLAGARISM REPORT

● 23% Overall Similarity

Top sources found in the following databases:

- 20% Internet database
- 10% Publications database
- Crossref database
- Crossref Posted Content database
- 15% Submitted Works database

TOP SOURCES

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	ijirt.org Internet	1%
2	github.com Internet	2%
3	ijsrem.com Internet	1%
4	Lili Li, Shujuan Zhang, Bin Wang. "Plant Disease Detection and Classifi..%2 Crossref	
5	Manipal University on 2022-09-25 Submitted works	2%
6	Wasswa Shafik, Ali Tufail, Abdallah Namoun, Liyanage Chandratilak De... Crossref	2%
7	Wasswa Shafik, Ali Tufail, Abdallah Namoun, Liyanage Chandratilak De... Crossref	2%
8	doaj.org Internet	2%

9	ijfmr.com Internet	2%
10	Joaquín Cañadas, Jorge Antonio Sánchez-Molina, Francisco Rodríguez... Crossref	2%
11	jetir.org Internet	1%
12	ijasi.org Internet	1%
13	Panipat Institute of Engineering & Technology on 2022-06-02 Submitted works	1%
14	coursehero.com Internet	1%
15	frontiersin.org Internet	1%
16	docshare.tips Internet	1%
17	The British College on 2022-01-27 Submitted works	1%
18	Madanapalle Institute of Technology and Science on 2015-04-24 Submitted works	<1%
19	ijcspub.org Internet	<1%
20	pantechsolutions.net Internet	<1%

21	publications.scrs.in Internet	<1%
22	St. Patrick's College on 2018-07-29 Submitted works	<1%
23	VNR Vignana Jyothi Institute of Engineering and Technology on 2022-... Submitted works	<1%
24	VNR Vignana Jyothi Institute of Engineering and Technology on 2023-... Submitted works	<1%
25	fastercapital.com Internet	<1%
26	Southern University And A & M College on 2024-04-30 Submitted works	<1%
27	ijartet.com Internet	<1%
28	wanivipin.files.wordpress.com Internet	<1%
29	studymoose.com Internet	<1%
30	Visvesvaraya Technological University, Belagavi on 2022-02-01 Submitted works	<1%
31	open-innovation-projects.org Internet	<1%
32	Bharati Vidyapeeth Deemed University College Of Engineering on 2014... Submitted works	<1%